

Latent Reasoning with Normalizing Flows

Guancheng Tu^{*,†}, Xiangjun Fu^{*,†}, Suhao Yu[†], Yao Tang[†], Haoqiang Kang[‡], Lianhui Qin[‡], Yizhe Zhang[§], Jiatao Gu[†]

[†]University of Pennsylvania, [‡]UC San Diego, [§]Meta

*Equal contribution

Abstract. Large language models often improve reasoning by generating explicit chain-of-thought (CoT), demonstrating the importance of intermediate computation. However, textual CoT forces this computation through a discrete, serial, and communication-oriented token stream: each reasoning step must be verbalized before the model can proceed, even when the underlying update is semantic, uncertain, or only partially formed. Latent reasoning offers a higher-bandwidth alternative by performing intermediate computation in compact continuous states before committing to text. Yet existing latent-reasoning methods often sacrifice key advantages that make CoT effective in autoregressive language models, including native left-to-right generation, probabilistic sampling, compatibility with KV-cache decoding, and tractable likelihood estimation. We propose NF-CoT, a latent reasoning framework that preserves these advantages by modeling continuous thoughts with normalizing flows. NF-CoT instantiates a TARFlow-style normalizing flow inside the LLM backbone, defining a tractable probability model over compact continuous thoughts distilled from explicit CoT. Continuous-thought positions are generated by an NF head, while text positions are generated by the standard LM head within the same causal stream. This design provides exact likelihoods for latent thoughts, enables probabilistic left-to-right decoding with the original KV cache, and supports direct policy-gradient optimization in the latent reasoning space. On code-generation benchmarks, NF-CoT improves pass rates over explicit-CoT and prior latent-reasoning baselines while substantially reducing intermediate-reasoning cost.

✉ **Correspondence:** {tgc, xjf, jgu32}@seas.upenn.edu 🏠 **Project Page:** <https://nf-cot.vercel.app>

1 Introduction

Chain-of-thought (CoT) prompting has become a standard chain-of-thoughtway to elicit reasoning in large language models (LLMs), improving performance by generating intermediate steps before the final answer (Wei et al., 2022; Kojima et al., 2022; Nye et al., 2021). One way to understand its effectiveness is that CoT introduces intermediate reasoning variables between the prompt and the answer, turning a direct input-output prediction into a conditioned prediction through a sampled reasoning path (Zelikman et al., 2022). In explicit CoT, these variables are represented as text tokens, making them naturally autoregressive, probabilistic, and likelihood-scored under the LLM. Yet text is a verbose, low-information-density medium for thought, making long reasoning costly and tying intermediate computation to surface forms (Hao et al., 2024).

This inefficiency has motivated latent CoT methods that replace textual reasoning traces with continuous or soft embedding states. Some methods keep reasoning close to the LLM by recycling hidden states or token-embedding mixtures as latent thoughts (Hao et al., 2024;

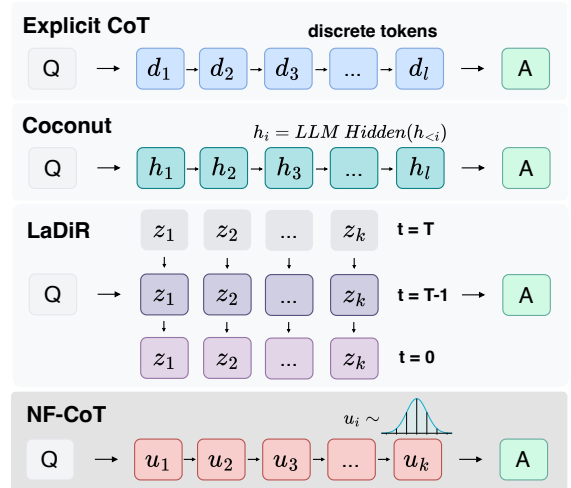


Figure 1 Four paradigms for chain-of-thought reasoning. **Explicit CoT:** *discrete* text tokens. **Coconut:** *deterministic* hidden states. **LaDiR:** *iteratively* denoised latents using diffusion. **NF-CoT (ours):** AR-sampled continuous thoughts.

Zhang et al., 2026b; Tang et al., 2026); they are efficient, but remain local and tied to the model’s token or embedding space. Others model stochastic continuous latents with diffusion models (Kang et al., 2025, 2026), but require iterative denoising and lack native left-to-right likelihood modeling. The challenge is to learn latent CoT without giving up the native sampling, scoring, and decoding interface of language modeling in current LLMs.

In this work, we address this gap by proposing NF-CoT, a latent reasoning framework that gives continuous CoT the same modeling status as language tokens. NF-CoT places a scalable normalizing flow (Zhai et al., 2024; Gu et al., 2026a) inside the LLM’s causal stream, using the LLM backbone as the shared autoregressive model for continuous thoughts and text answers. During training, NF-CoT turns explicit CoT supervision into continuous thoughts and optimizes an end-to-end likelihood objective over both latent reasoning and answers. The resulting tractable likelihood also enables policy-gradient optimization (Guo et al., 2025) directly in the continuous reasoning space. At inference, NF-CoT samples continuous thoughts left-to-right and decodes answers in a single causal pass. Together, this preserves the sampling, decoding, and likelihood interface of explicit CoT in a compact continuous space, at lower token cost.

We evaluate NF-CoT on MBPP, MBPP+, HumanEval, HumanEval+, and LiveCodeBench v6. NF-CoT improves pass rates over explicit-CoT and prior latent-reasoning baselines while substantially reducing intermediate-reasoning cost. We further study likelihood-based sampling and policy-gradient optimization in the continuous reasoning space. To this end, our contributions are threefold:

- A likelihood-based formulation of continuous CoT that preserves the sampling, scoring, and decoding interface of explicit CoT;
- A novel architecture that jointly models latent thoughts and text answers with NF and LM heads over a shared LLM backbone;
- A unified optimization interface for continuous thoughts, enabling supervised likelihood training and policy-gradient refinement.

2 Preliminaries

2.1 Chain-of-Thought Reasoning

We consider supervised reasoning examples with a prompt q , a target answer sequence $x_{1:N}$, and, when available, an explicit chain-of-thought (CoT) trace $d_{1:L}$. A probabilistic view treats the trace as an intermediate variable:

$$p_{\theta}(x_{1:N} \mid q) = \sum_{d_{1:L}} p_{\theta}(d_{1:L} \mid q) p_{\theta}(x_{1:N} \mid q, d_{1:L}). \quad (2.1)$$

Since $d_{1:L}$ is a text sequence, it can be generated, sampled, and scored with the same autoregressive language-modeling interface as the answer. This is convenient but forces reasoning into verbose natural-language tokens at high token cost.

2.2 Continuous Chain-of-Thought

Continuous CoT replaces text rationales with continuous codes $e_{1:K} \in \mathbb{R}^{K \times D}$ of length K and dimension D , which play the role of the intermediate variable in Eq. (2.1).

Coconut (Hao et al., 2024) distills reasoning into hidden states fed back autoregressively, but the resulting thoughts are deterministic and do not define a distribution over reasoning paths. Another approach is to first learn a continuous code space with a VAE: an encoder maps $d_{1:L}$ together with K learnable queries to $q_{\phi}(e_{1:K} \mid d_{1:L})$, trained with

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_{\phi}(e \mid d)} [-\log p_{\psi}(d \mid e)] + \beta \text{KL}(q_{\phi}(e \mid d) \parallel \mathcal{N}(0, I)). \quad (2.2)$$

After pretraining, a separate generative model (e.g., diffusion (Kang et al., 2025)) is trained for $p_{\theta}(e_{1:K} \mid q)$. This gives a stochastic continuous representation but separates latent-thought generation from the native autoregressive likelihood interface used by explicit CoT.

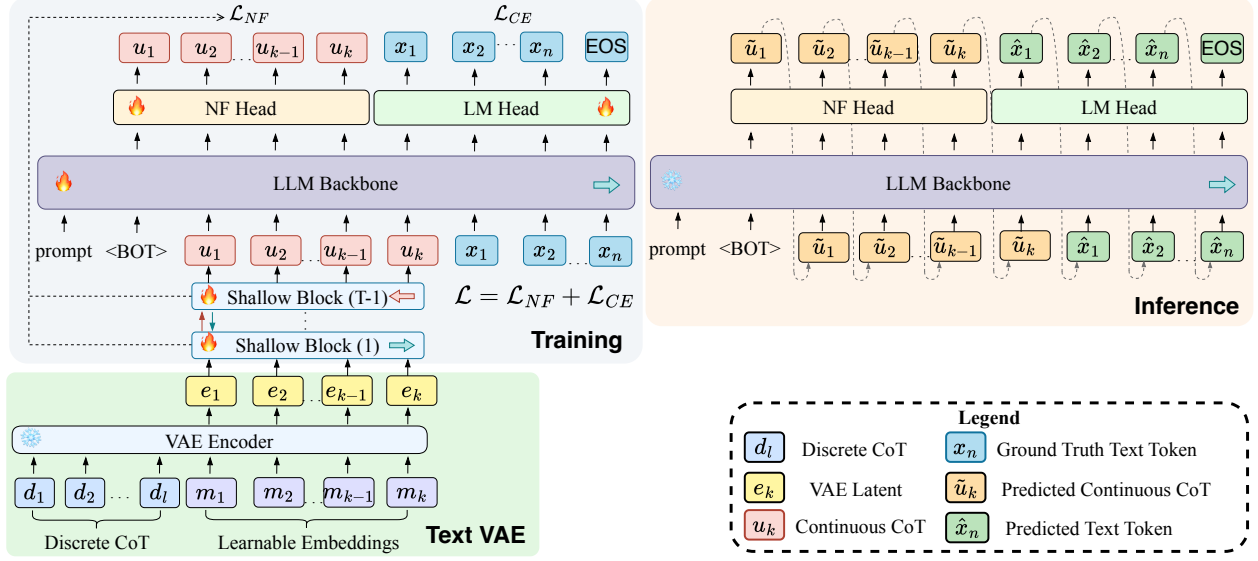


Figure 2 Training and inference pipeline of NF-CoT. Training uses a frozen VAE encoder and shallow flow blocks to convert discrete-CoT rationales into continuous-CoT states $u_{1:k}$. A shared LLM then consumes the prompt, $\langle \text{BOT} \rangle$, continuous-CoT states, and ground-truth text tokens $x_{1:n}$ in one causal pass, jointly optimizing the flow NLL and answer cross-entropy. At inference, the VAE branch and shallow blocks are skipped: the LLM samples predicted continuous-CoT states $\tilde{u}_{1:k}$ left-to-right and continues decoding predicted text tokens $\hat{x}_{1:n}$ with the same KV cache, without recomputing the prompt or CoT prefix.

2.3 Scalable Normalizing Flows

Normalizing flows (Dinh et al., 2014, 2016) map data y to a simple base variable $z \sim \mathcal{N}(0, I)$ through an invertible network. For a conditional flow with context c ,

$$z = f_{\theta}^{-1}(y; c), \quad p_{\theta}(y | c) = p(z) \left| \det J_{f_{\theta}^{-1}}(y; c) \right|, \quad (2.3)$$

giving exact likelihood evaluation and direct sampling. Autoregressive flows (Kingma et al., 2016; Papamakarios et al., 2017) transform each position using only the context and previous positions, yielding a triangular Jacobian compatible with causal sequence modeling. Scalable variants such as TarFlow (Zhai et al., 2024) and STARFlow (Gu et al., 2026a) use Transformer blocks for high-dimensional structured latents. We build on this family to model continuous CoT with LLMs.

3 Method

3.1 Formulation

We start from the continuous CoT target $e_{1:K}$ defined in Sec. 2.2. Since this encoder-derived space is optimized for text compression rather than autoregressive generation, its prompt-conditioned distribution can be difficult to model directly. Prior work thus turns to diffusion models (Kang et al., 2025), trading expressive latent modeling for iterative sampling and a less direct likelihood interface.

NF-CoT takes a different route. We introduce an LLM-facing continuous thought space designed to make continuous thoughts easy to sample and score autoregressively. Following STARFlow (Gu et al., 2026a), we model continuous thought tokens $u_{1:K}$ with a causal Gaussian density:

$$p_{\theta}(u_{1:K} | q) = \prod_{i=1}^K \mathcal{N}(u_i; \mu_{\theta}(q, u_{<i}), \text{diag}(\sigma_{\theta}^2(q, u_{<i}))). \quad (3.1)$$

where μ_{θ} and σ_{θ} are causal functions of the prompt and previous thoughts. This space supports natural left-to-right sampling with tractable likelihood.

The two spaces are connected through an invertible transformation $u_{1:K} = F_\theta(e_{1:K}; q)$, implemented as a stack of shallow autoregressive flow blocks following Gu et al. (2026a). Since F_θ is invertible, $u_{1:K}$ is information-equivalent to $e_{1:K}$ rather than a lossy compression. The full likelihood of the original continuous CoT target can therefore be written through the reparameterized thought space:

$$\log p_\theta(e_{1:K} | q) = \log p_\theta(u_{1:K} | q) + \log |\det J_{F_\theta}(e_{1:K}; q)|. \quad (3.2)$$

This allows the shallow flow blocks and the autoregressive density over $u_{1:K}$ to be trained end-to-end under an exact likelihood. Thus, NF-CoT learns from the same continuous CoT targets as prior latent methods, but performs modeling and generation in an information-equivalent thought space that is easier to sample and score autoregressively.

3.2 Architecture

Architecturally, NF-CoT contains two flow components, as shown in Fig. 2: shallow flow blocks for reparameterizing e into u , followed by an LLM-parameterized deep autoregressive flow over u . The shallow blocks implement the invertible map $F_\theta : e_{1:K} \mapsto u_{1:K}$ and are trained from scratch with identity initialization ($F_\theta \approx I$). Because they are causal affine flows, their Jacobian is triangular and their log-determinant is tractable, contributing to the likelihood in Eq. (3.2).

The deep flow is implemented within the LLM backbone. Continuous thoughts $u_{1:K}$ are projected into the token embedding dimension and processed in the same causal stream as answer tokens, in the spirit of interleaved continuous-discrete generation (Shen et al., 2026). At continuous-thought positions, an NF head maps the hidden state to the parameters $\mu_\theta(q, u_{<i})$ and $\sigma_\theta(q, u_{<i})$ of the conditional density in Eq. (3.1). At answer positions, the standard LM head maps the hidden state to token logits. Thus, latent reasoning and answer generation share one causal backbone with separate heads for continuous and discrete variables. This unified design is the main architecture of NF-CoT; we report a dual-path variant, separating the latent-flow and answer-generation paths, as an ablation.

3.3 Learning

Supervision. We train NF-CoT from supervised reasoning examples with prompt q , explicit CoT trace $d_{1:L}$ when available, and answer $x_{1:N}$. As described in Sec. 2.2, we first pretrain a CoT encoder, freeze it, and use its posterior mean as the continuous CoT target $e_{1:K}$. The trainable model maps this target to the LLM-facing thought sequence $u_{1:K} = F_\theta(e_{1:K}; q)$ and learns to model both the latent reasoning path and the final answer.

Unified likelihood objective. We train latent reasoning and answer generation with a single supervised objective. The flow term assigns exact likelihood to the original continuous CoT target, $\mathcal{L}_{\text{flow}} = -\log p_\theta(e_{1:K} | q)$, computed through the reparameterized thought space using Eq. (3.2). The text term is the standard autoregressive likelihood conditioned on the continuous thoughts, $\mathcal{L}_{\text{text}} = -\sum_{j=1}^N \log p_\theta(x_j | q, u_{1:K}, x_{<j})$. The final objective is

$$\mathcal{L}_{\text{sup}} = \lambda_{\text{flow}} \mathcal{L}_{\text{flow}} + \lambda_{\text{text}} \mathcal{L}_{\text{text}}. \quad (3.3)$$

Both terms are likelihoods under the same causal LLM: the flow head scores continuous thoughts, while the LM head scores answer tokens.

Training curriculum. We use a two-stage curriculum. First, we freeze the LLM backbone and train only the shallow flow blocks and continuous-thought projection layers with the same supervised objective in Eq. (3.3), aligning the $e \rightarrow u$ reparameterization with the frozen LLM space. We then unfreeze all parameters and continue end-to-end training with the full objective.

Following the practice in TarFlow (Zhai et al., 2024), we add small Gaussian noise to $e_{1:K}$ before computing the flow loss. We find that this improves robustness to deterministic target codes.

3.4 Inference

At inference time, NF-CoT does not run the shallow flow blocks F_θ , which are only needed during training. Instead, inference proceeds directly in u -space: given a prompt q , the model samples continuous thoughts left-to-right from the autoregressive density in Eq. (3.1). This sampling is stochastic, analogous to token sampling in a standard LLM, rather than a deterministic hidden-state update. After sampling $\tilde{u}_{1:K}$, the model switches to the LM head and continues answer generation in the same causal stream. Because continuous thoughts and answer tokens share the LLM backbone, the KV cache built during thought sampling is reused for answer generation at no extra cost.

This inference procedure differs from prior latent-CoT methods in two ways. First, NF-CoT samples from an explicit likelihood model over continuous thoughts rather than deterministically recycling hidden states. Second, it avoids diffusion-style iterative latent generation: although training uses the invertible map between e and u , test-time reasoning happens directly in the information-equivalent u -space, which is shaped for autoregressive sampling and decoding.

3.5 Reinforcement Learning

After supervised training, we further refine NF-CoT with reinforcement learning. During this stage, we freeze the shallow flow blocks and optimize the policy directly in the LLM-facing thought space u . For a sampled thought-answer trajectory $(\tilde{u}_{1:K}, \hat{x}_{1:\hat{N}})$, the policy likelihood decomposes as

$$\log \pi_\theta(\tilde{u}, \hat{x} \mid q) = \log p_\theta(\tilde{u}_{1:K} \mid q) + \log p_\theta(\hat{x}_{1:\hat{N}} \mid q, \tilde{u}_{1:K}). \quad (3.4)$$

The first term is the Gaussian autoregressive likelihood from the NF head, and the second term is the token likelihood from the LM head. This makes policy-gradient optimization directly applicable to both continuous thoughts and text answers.

In our experiments, we use a GRPO-style objective over groups of sampled trajectories. Given a reward $R(q, \hat{x})$, such as unit-test feedback for code generation, we compute a group-normalized advantage $A(q, \tilde{u}, \hat{x})$ and update the policy via the policy gradient

$$\nabla_\theta J_{\text{RL}} = \mathbb{E} [A(q, \tilde{u}, \hat{x}) \nabla_\theta \log \pi_\theta(\tilde{u}, \hat{x} \mid q)]. \quad (3.5)$$

Since the shallow blocks are frozen, this refinement is equivalent to applying policy gradients to the Gaussian likelihood over u together with the token likelihood over answers in a single update.

4 Experiments

We evaluate NF-CoT on code generation, a reasoning-intensive task with standard correctness metrics. We compare with strong baselines, including autoregressive CoT models, diffusion-based latent reasoners, and recent continuous-reasoning methods. All methods use the same setup with pass@1 and pass@k on multiple benchmarks.

4.1 Experimental Setup

Datasets and evaluation. We fine-tune NF-CoT on Ling-Coder (Team et al., 2025), a dataset of 1.4M Python instruction-following samples. We evaluate on five Python code benchmarks: HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), their EvalPlus versions HumanEval+ and MBPP+ (Liu et al., 2023), and LiveCodeBench v6 (Jain et al., 2025). HumanEval+ and MBPP+ add more test cases for stricter evaluation. For each problem, we sample 16 solutions and report the average pass@1 to reduce variance. For the RL stage, we further train on data from AceCoder (Zeng et al., 2025) and KodCode (Xu et al., 2025).

Baselines. We compare NF-CoT with strong open-source models and latent reasoning baselines. Autoregressive baselines include Qwen2.5-Coder (Hui et al., 2024), OpenCoder (Huang et al., 2025), OlympicCoder (Hugging Face, 2025), and Seed-Coder (Seed et al., 2025); diffusion baselines (Ho et al., 2020) include Dream (Ye et al., 2025), LLaDA (Nie et al., 2026), Diffu-Coder (Gong et al., 2025), Dream-Coder (Xie et al., 2025), and

Table 1 Pass@1 results on code generation benchmarks. Latent reasoning methods are highlighted in light yellow. Improvements over the base model are shown in green, and decreases in red. Best results across all reported methods are in bold and second-best results are underlined.

Model / Method	Base Model	MBPP	MBPP+	HumanEval	HumanEval+	LCB v6	Avg.
Open-Sourced Models							
Autoregressive Coding Models							
Qwen 2.5 Coder*	Qwen2.5-7B	75.9	62.9	66.5	60.4	26.8	59.9
OpenCoder*	from scratch (8B)	79.9	<u>70.4</u>	66.5	63.4	29.6	62.0
OlympicCoder*	Qwen2.5-7B	80.0	66.4	82.1	76.9	<u>37.3</u>	<u>68.5</u>
Seed-Coder*	from scratch	<u>82.0</u>	69.0	77.4	68.3	28.4	65.0
Diffusion Language Models							
Dream*	Qwen2.5-7B	68.7	57.4	56.7	50.0	18.6	—
LLaDA*	from scratch (8B)	50.1	42.1	35.4	30.5	12.4	—
Diffu-Coder*	Qwen2.5-Coder-7B	75.1	61.9	72.0	65.2	24.5	59.7
Dream-Coder*	Qwen2.5-Coder-7B	75.9	61.6	66.5	60.4	21.4	57.2
d1*	LLaDA+RL	39.0	—	45.5	—	—	—
Looped Latent Reasoning Models							
Ouro*	from scratch (2.6B)	80.4	66.6	78.2	70.7	38.7	66.9
Method Comparison							
Non-RL Methods							
Base Model	Qwen3-8B-Base	60.5	53.8	78.2	68.6	17.7	55.8
Standard SFT	Qwen3-8B-Base	63.3+2.8	52.7-1.1	84.6+6.4	69.5+0.9	—	—
Soft Thinking	Qwen3-8B-Base	64.2+3.7	53.1-0.7	85.0+6.8	71.2+2.6	—	—
TaH+	Qwen3-8B-Base	65.6+5.1	56.5+2.7	85.8+7.6	74.3+5.7	—	—
LaVAE	Qwen3-8B-Base	42.0-18.5	30.2-23.6	47.8-30.4	32.8-35.8	10.8-6.9	32.7-23.0
LaDiR	Qwen3-8B-Base	66.8+6.3	59.5+5.7	87.4 +9.2	73.2+4.6	21.3+3.6	61.6+5.9
NF-CoT (Dual-Path)	Qwen3-8B-Base	77.5+17.0	66.2+12.4	82.9+4.7	<u>77.7</u> +9.1	21.6+3.9	65.2+9.4
NF-CoT (Unified)	Qwen3-8B-Base	83.9 +23.4	72.1 +18.3	<u>85.8</u> +7.6	78.3 +9.7	23.7+6.0	68.8 +13.0
↪ RL	Qwen3-8B-Base	85.4+24.9	73.3+19.5	86.7+8.5	80.2+11.6	25.1+7.4	70.1+14.3

d1 (Zhao et al., 2026); we also include the looped latent reasoning model Ouro (Zhu et al., 2025). Under the same Qwen3-8B-Base backbone (Yang et al., 2025), we further compare with Standard SFT, Soft Thinking (Zhang et al., 2026b), TaH+ (Fu et al., 2025), LaVAE (replaces our flow with a L_2 objective VAE) and LaDiR (Kang et al., 2025) (replaces our flow with latent diffusion). We report two variants of our method, NF-CoT (Dual-Path) and NF-CoT (Unified).

Experiment details. We use Qwen3-8B-Base with $N = 64$ latent slots and a flow stack of $K = 5$ shallow MetaBlocks (alternating Identity/Flip permutations) followed by the deep LLM block. Full training and inference details are in Appendix B.

4.2 Main Results

Pass@1 improvements on code generation. Table 1 reports pass@1 on five code benchmarks. NF-CoT (Unified) achieves the best average performance, improving Qwen3-8B-Base from 55.8 to 68.8 (+13.0%), while NF-CoT (Dual-Path) also improves the base by +9.4%. On the four shared non-LCB benchmarks, Standard SFT improves the base from 65.3 to 67.5, while NF-CoT (Unified) reaches 80.0 with the same training data. This suggests that the gains come from modeling the distribution of reasoning trajectories, not simply from more CoT data. Among prior latent reasoning methods, NF-CoT (Unified) outperforms the strongest baseline LaDiR with a +7.1% gain on average. This shows that learning a continuous CoT distribution with normalizing flow captures latent reasoning structure more effectively. NF-CoT (Unified) also surpasses the strongest open-source baselines OlympicCoder (+0.3%), Diffu-Coder (+9.1%), and the looped latent reasoning model Ouro (+1.9%) in average pass@1.

Pass@ k scaling We test whether latent reasoning scales with more samples by comparing NF-CoT with the Qwen3-8B base model and LaDiR on MBPP+ and HumanEval+ for k up to 128.

Figure 3 shows that NF-CoT dominates both baselines across the full k range in this separate scaling run.

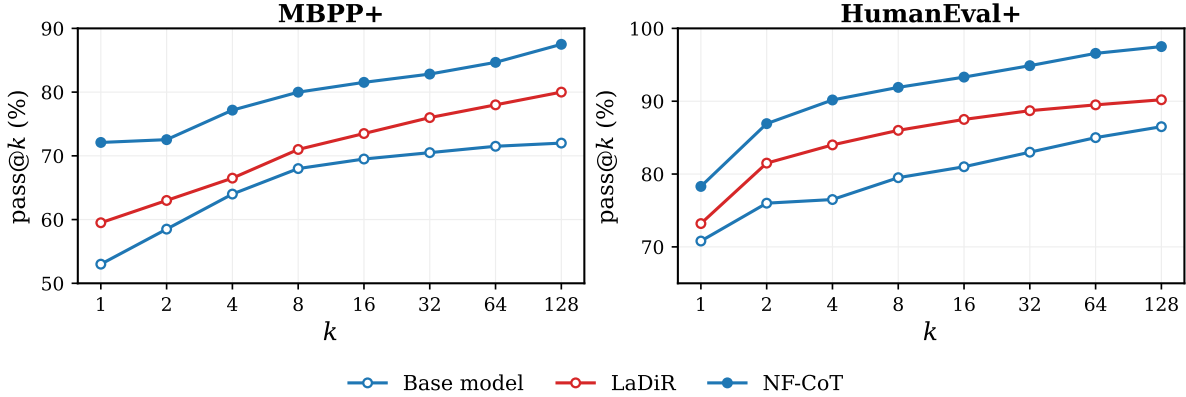


Figure 3 Pass@ k scaling on MBPP+ and HumanEval+. NF-CoT outperforms the base model and LaDiR, and continues to improve with larger k .

On MBPP+, NF-CoT’s pass@1 (72.1) already matches the base model’s pass@128 (72.0) and rises to 87.5 at $k = 128$. On HumanEval+, NF-CoT improves from 78.3 to 97.5 (+19.2), comparable to the base model’s +16.5 despite starting from a much higher pass@1. LaDiR, in contrast, only improves from 73.2 to 90.2 (+17.0), with the curve flattening at larger budgets. This suggests that sampling the flow noise produces distinct continuous-CoT trajectories rather than collapsing onto a single solution mode. We further verify this with pairwise structural similarity among passing programs in Appendix C.2, confirming structural diversity. Qualitative decoded-latent examples in Appendix A provide example-level evidence for the same phenomenon: different latent samples for the same HumanEval prompt can steer decoding toward distinct correct implementation strategies. We treat these decoded latent CoTs only as qualitative probes, not as faithful natural-language explanations of the model’s internal reasoning process.

Execution-guided latent RL. We further apply GRPO to the supervised NF-CoT (Unified) using execution rewards on a random selected 20K-problem Python mixture for 150 training steps. As shown in Table 1, this lightweight RL stage consistently improves NF-CoT across all five benchmarks, raising the average from 68.8 to 70.1. These modest gains show that the tractable likelihood of NF-CoT provides a direct interface for policy optimization in latent space, allowing execution rewards to update not only the answer decoder but also the distribution over latent reasoning paths. RL setup details are in Appendix B.5.

RL preserves pass@ k diversity. A common concern for reinforcement learning on autoregressive language models is that optimizing a single correctness reward can increase pass@1 while reducing the diversity needed for pass@ k scaling. We therefore compare standard token-space GRPO with our latent-space RL under the same pass@ k diagnostic on MBPP+ and HumanEval+. As shown in Figure 4, the two RL settings exhibit different large- k behavior. In the token-space setting, AR Vanilla GRPO improves the low- k region but saturates below, or only matches, the base model at large k . This indicates that token-space RL can concentrate probability mass on a smaller set of solution modes: the model becomes more likely to find one correct solution, but additional samples do not provide substantially broader coverage.

In contrast, applying RL to NF-CoT does not produce this pass@ k collapse. NF-CoT + RL improves pass@1 while preserving the upward scaling trend at larger sampling budgets, and it remains consistently above the supervised NF-CoT checkpoint across the full k range on both benchmarks. This suggests that policy-gradient refinement in the learned continuous-CoT space improves solution quality without collapsing the latent trajectory distribution. Together with the exact latent likelihood in Eq. (3.4), this supports the role of NF-CoT as an RL-compatible latent reasoning policy: execution rewards can refine both answer decoding and latent trajectory sampling, while maintaining the diversity required for pass@ k gains.

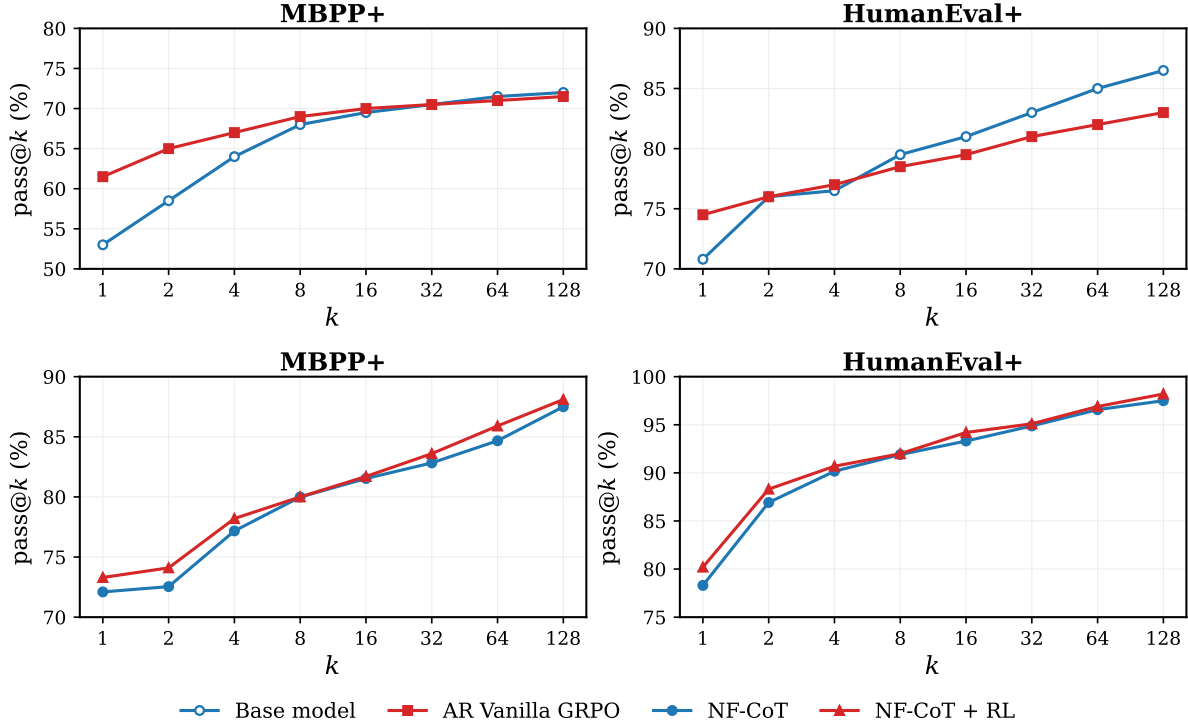


Figure 4 Pass@k diversity before and after reinforcement learning on MBPP+ and HumanEval+. Top row: standard token-space GRPO improves the low- k region but saturates at larger k , failing to provide the same large-sample coverage as the base model. Bottom row: latent-space RL for NF-CoT improves pass@1 while preserving the upward pass@k scaling trend, indicating that policy-gradient refinement in continuous-CoT space does not collapse the latent trajectory distribution.

Table 2 HumanEval inference efficiency with vLLM answer decoding. All methods generate 16 candidates per problem; LaDiR uses 30 denoising steps. “Latent” and “Decode” report latent-generation and answer-decoding time, respectively. FLOPs/sample are coarse estimates from saved outputs.

Method	Latent (s) ↓	Decode (s) ↓	Total (s) ↓	Samples/s ↑	FLOPs/sample ↓
NF-CoT (Unified)	173.5	152.1	325.6	8.06	19.9T
NF-CoT (Dual-Path)	232.3	147.7	380.0	6.90	21.6T
LaDiR	468.2	157.1	625.3	4.20	49.3T

5 Analysis

5.1 Training and inference efficiency

Tables 2 and 3 compare NF-CoT with LaDiR on HumanEval under matched settings (16 candidates per problem, Qwen3-8B backbone, vLLM decoding). Both methods represent reasoning using only 64 latent tokens, which on average encode a short CoT of 385 text tokens, corresponding to an effective compression rate of approximately $6.0\times$ compared to the explicit-CoT method. The key difference is in latent generation: LaDiR runs 30 denoising steps over the latent trajectory, while NF-CoT samples 64 continuous thoughts autoregressively in one left-to-right pass. This makes NF-CoT (Unified) $2.70\times$ faster at latent generation ($468.2 \rightarrow 173.5$ s), $1.92\times$ faster overall ($625.3 \rightarrow 325.6$ s), and $2.48\times$ cheaper in per-sample compute ($49.3 \rightarrow 19.9$ T FLOPs). The dual-path variant is slower than Unified because it keeps separate objective-specific paths, but still $1.65\times$ faster and $2.29\times$ cheaper than LaDiR. The same trend holds during training: NF-CoT (Unified) achieves $2.85\times$ sample and $5.71\times$ token throughput over LaDiR, with $6.66\times$ fewer total FLOPs; NF-CoT (Dual-Path) also reaches $1.90\times$ higher sample throughput. These results support our efficiency claim: replacing iterative denoising with an autoregressive flow speeds up latent generation, while exact-likelihood

Table 3 Stage-2 training throughput and compute. Note that LaDiR uses its paper setting of 30 denoising steps.

Method	Samples/s $\uparrow$	Tokens/s $\uparrow$	Total FLOPs $\downarrow$
NF-CoT (Unified)	18.4	5.88K	2.25e19
NF-CoT (Dual-Path)	12.2	3.91K	2.25e19
LaDiR	6.45	1.03K	1.50e20

Table 4 Ablation of the frozen-backbone warm-up. Removing Stage 1 hurts three benchmarks (pass@1).

Benchmark	Stage 1+2	Stage-2-only	Δ
HumanEval	84.4	81.5	-2.9
HumanEval+	78.7	75.5	-3.2
LiveCodeBench v6	23.1	21.4	-1.7

training avoids learning a multi-step denoising chain required by diffusion-based methods such as LaDiR.

5.2 Ablation study

Effect of the frozen-backbone warm-up stage. We ablate the two-stage training curriculum from Section 3. In the default setting, NF-CoT first trains the flow components with the LLM backbone frozen, then unfreezes all parameters for joint optimization. We compare this with a stage-2-only variant that skips the warm-up. As shown in Table 4, removing Stage 1 consistently hurts performance: HumanEval drops from 84.4 to 81.5, HumanEval+ from 78.7 to 75.5, and LiveCodeBench v6 from 23.1 to 21.4. This is not due to worse final optimization: both runs reach nearly the same total loss and answer cross-entropy. The difference appears in the early training dynamics. With the warm-up, the flow components already provide a meaningful latent interface before the backbone is updated; without it, the backbone receives early gradients from randomly initialized flow components, which can disrupt the coding capability we aim to preserve. The warm-up thus acts as a curriculum that aligns the latent-flow interface with a frozen backbone before allowing the NF likelihood objective to update the full model. Appendix C.1 provides additional backbone-drift diagnostics.

Unified versus dual-path training. We next ablate the unified causal training design. The dual-path variant uses separate paths for the NF and CE objectives: one path trains the NF head on continuous thoughts, the other trains the LM head to decode answers from a different latent interface. Although the two paths share the backbone, they use different prefix tokens, projections, and parameterizations, creating a mismatch between the latent distribution learned by NF, the latent representation used for answer conditioning, and the trajectory used at inference. The unified variant removes this mismatch by placing latent reasoning and answer decoding in a single causal sequence: the same continuous-CoT prefix is scored by the NF head and then used by the LM head for answer prediction, so the NF and CE losses are defined on the same context and the same trajectory used at inference. This structural alignment explains why NF-CoT (Unified) outperforms the dual-path variant: the gain comes from removing redundant latent interfaces and training continuous-CoT states to be useful for answer decoding, avoiding a second backbone pass and prefix recomputation.

5.3 Latent Perturbation Robustness

We test whether NF-CoT learns a smooth latent reasoning space. For each HumanEval problem, we sample a base continuous-CoT trajectory $\tilde{u}$, perturb it with Gaussian noise

$$u_\sigma = \tilde{u} + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (5.1)$$

and greedily decode from the perturbed prefix, measuring pass@1, pairwise pass agreement, exact-text match, cosine similarity, and the canonical-solution log-probability. As σ grows from 0 to 3.0 (Figure 5), cosine similarity drops from 1.000 to 0.116, but pass@1 only decreases from 86.0 to 83.6 with overlapping confidence intervals, indicating that the learned continuous-CoT space is locally smooth. Perturbations affect form much more than function: pairwise pass agreement drops only slightly ($1.000 \rightarrow 0.888$), while exact-text match drops sharply ($0.973 \rightarrow 0.278$). Continuous-CoT thus acts less like brittle hidden tokens and more

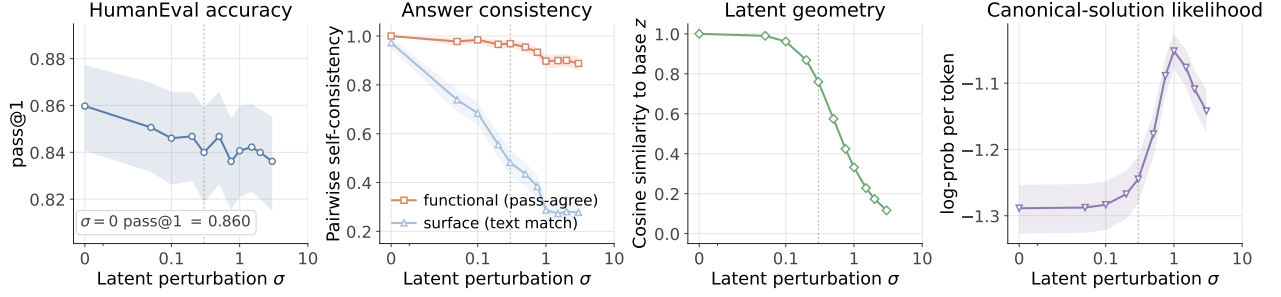


Figure 5 Effect of continuous-CoT perturbation strength on HumanEval generation. As σ increases, the perturbed continuous-CoT trajectory becomes nearly orthogonal to the original trajectory, and exact-text match drops sharply. Nevertheless, pass@1 remains nearly flat, indicating that local perturbations change the form of the generated program much more than its functional correctness, even at the largest perturbation strengths tested.

like distributed control variables over solution trajectories, perturbations switch implementation style rather than break correctness. The canonical-solution log-probability follows a non-monotonic trend: moderate noise raises it (the original trajectory commits to one valid solution mode that may differ from the canonical implementation), while large noise degrades the prefix. Overall, NF-CoT’s latent variables control which solution the model follows, not whether it solves the problem.

6 Related Work

CoT and latent reasoning. Chain-of-thought (CoT) prompting elicits reasoning by generating intermediate steps before the answer (Wei et al., 2022; Kojima et al., 2022), with further gains from supervised or RL-based training (Zelikman et al., 2022; Shao et al., 2024) and test-time strategies that sample and score many chains (Wang et al., 2022; Snell et al., 2024; Yao et al., 2023). These approaches all reason in token space. Latent reasoning instead feeds continuous thoughts back into the model (Hao et al., 2024; Zhang et al., 2026b; Tang et al., 2026; Su et al., 2025), but probing shows each collapses to a single greedy path, not a superposition (Wu et al., 2025). Recent work injects stochasticity to enable RL or model trajectory superpositions, including in the multimodal setting (Butt et al., 2025; Zhou et al., 2026; Deng et al.; Wang et al., 2026), several via Gumbel-Softmax relaxations that turn token choices into soft embedding mixtures (Jang et al., 2016; Maddison et al., 2016), but none place a tractable density over the trajectories. Others run multiple latent channels or internal recurrences without modeling a trajectory distribution (Geiping et al., 2026; Zhu et al., 2025; Fu et al., 2025). The closest precedent denoises a VAE latent space, leaving the trajectory density implicit and the log-likelihood intractable (Kang et al., 2025, 2026). NF-CoT instead uses an autoregressive normalizing flow over VAE latents inside the LLM’s causal stream, giving an exact log-likelihood over continuous-CoT trajectories and training latent reasoning and answer generation jointly.

Normalizing flows. Normalizing flows map a simple base distribution to data through invertible networks, and provide exact, tractable likelihoods (Rezende and Mohamed, 2015). Early work applied them to images, showing that flows can scale to natural image generation (Dinh et al., 2014, 2016; Kingma and Dhariwal, 2018). A parallel line adapted flows to NLP by lifting discrete text into continuous latent spaces, with applications to language modeling and non-autoregressive translation (Ziegler and Rush, 2019; Ma et al., 2019). Autoregressive flows (Kingma et al., 2016; Papamakarios et al., 2017), whose triangular Jacobian aligns naturally with causal Transformers, have more recently been scaled to high-resolution images and video (Zhai et al., 2024; Gu et al., 2026a, 2025, 2026b) and, most recently, to text, either by unifying image and text generation or by modeling language as a continuous latent sequence (Shen et al., 2026; Zhang et al., 2026a). In contrast, NF-CoT uses autoregressive flows inside a pretrained LLM as a tool for *reasoning*, modeling a distribution over continuous CoT trajectories.

7 Conclusion

We presented NF-CoT, a latent reasoning framework that gives continuous CoT the same modeling status as language tokens by running an autoregressive normalizing flow inside the LLM’s causal stream. NF-CoT models an explicit distribution over reasoning trajectories with exact likelihood, which supports both supervised likelihood training and policy-gradient refinement in the continuous reasoning space. Across code generation benchmarks, NF-CoT improves accuracy over SFT and prior latent-reasoning baselines on Qwen3-8B-Base, and runs faster than LaDiR in training and inference. These results suggest that likelihood-based latent reasoning offers a practical interface for sampling, scoring, and refining continuous thoughts in LLMs.

Limitations

Our validation focuses on code-generation benchmarks; extending to other reasoning tasks remains future work. NF-CoT is trained on fixed-length VAE-encoded trajectories from explicit-CoT traces, so the learned density inherits their biases, and a fixed latent budget may not fit all problems. Latent likelihood is not a correctness oracle, and our RL refinement relies on unit-test rewards available only in code; generalization to domains without verifiers is open. Improved code generation may also produce plausible-yet-incorrect or unsafe code, and continuous latents are not human-readable, so decoded CoTs are qualitative probes rather than faithful explanations of internal reasoning.

References

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Natasha Butt, Ariel Kwiatkowski, Ismail Labiad, Julia Kempe, and Yann Ollivier. Soft tokens, hard truths. *arXiv preprint arXiv:2509.19170*, 2025.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Jingcheng Deng, Liang Pang, Zihao Wei, Shicheng Xu, Zenghao Duan, Kun Xu, Yang Song, Huawei Shen, and Xueqi Cheng. Llm latent reasoning as chain of superposition, 2026. URL <https://arxiv.org/abs/2510.15522>.
- Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.
- Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real nvp. *arXiv preprint arXiv:1605.08803*, 2016.
- Tianyu Fu, Yichen You, Zekai Chen, Guohao Dai, Huazhong Yang, and Yu Wang. Think-at-hard: Selective latent iterations to improve reasoning language models. *arXiv preprint arXiv:2511.08577*, 2025.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *Advances in Neural Information Processing Systems*, 38:41340–41391, 2026.
- Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. Diffu-coder: Understanding and improving masked diffusion models for code generation. *arXiv preprint arXiv:2506.20639*, 2025.
- Jiatao Gu, Ying Shen, Tianrong Chen, Laurent Dinh, Yuyang Wang, Miguel Angel Bautista, David Berthelot, Josh Susskind, and Shuangfei Zhai. Starflow-v: End-to-end video generative modeling with normalizing flows. *arXiv preprint arXiv:2511.20462*, 2025.
- Jiatao Gu, Tianrong Chen, David Berthelot, Huangjie Zheng, Yuyang Wang, Ruixiang Zhang, Laurent Dinh, Miguel Angel Bautista, Joshua Susskind, and Shuangfei Zhai. Starflow: Scaling latent normalizing flows for high-resolution image synthesis. *Advances in Neural Information Processing Systems*, 38:120986–121022, 2026a.

- Jiatao Gu, Tianrong Chen, Ying Shen, David Berthelot, Shuangfei Zhai, and Josh Susskind. Normalizing trajectory models. *arXiv preprint arXiv:2605.08078*, 2026b.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081): 633–638, 2025.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Siming Huang, Tianhao Cheng, Jason Klein Liu, Weidi Xu, Jiaran Hao, Liuyihan Song, Yang Xu, Jian Yang, Jiaheng Liu, Chenchen Zhang, et al. Opencoder: The open cookbook for top-tier code large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33167–33193, 2025.
- Hugging Face. Open r1: A fully open reproduction of deepseek-r1, January 2025. URL <https://github.com/huggingface/open-r1>.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pages 58791–58831, 2025.
- Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- Haoqiang Kang, Yizhe Zhang, Nikki Lijing Kuang, Nicklas Majamaki, Navdeep Jaitly, Yi-An Ma, and Lianhui Qin. Ladir: Latent diffusion enhances llms for text reasoning. *arXiv preprint arXiv:2510.04573*, 2025.
- Haoqiang Kang, Yizhe Zhang, Nikki Lijing Kuang, Yi-An Ma, and Lianhui Qin. Beyond mode elicitation: Diversity-preserving reinforcement learning via latent diffusion reasoner. *arXiv preprint arXiv:2602.01705*, 2026.
- Durk P Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. *Advances in neural information processing systems*, 31, 2018.
- Durk P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. *Advances in neural information processing systems*, 29, 2016.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in neural information processing systems*, 36:21558–21572, 2023.
- Xuezhe Ma, Chunting Zhou, Xian Li, Graham Neubig, and Eduard Hovy. Flowseq: Non-autoregressive conditional sequence generation with generative flow. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4282–4292, 2019.
- Chris J Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. *arXiv preprint arXiv:1611.00712*, 2016.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *Advances in Neural Information Processing Systems*, 38:50608–50646, 2026.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. Show your work: Scratchpads for intermediate computation with language models. 2021.
- George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation. *Advances in neural information processing systems*, 30, 2017.

- Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR, 2015.
- ByteDance Seed, Yuyu Zhang, Jing Su, Yifan Sun, Chenguang Xi, Xia Xiao, Shen Zheng, Anxiang Zhang, Kaibo Liu, Daoguang Zan, et al. Seed-coder: Let the code model curate data for itself. *arXiv preprint arXiv:2506.03524*, 2025.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Ying Shen, Tianrong Chen, Yuan Gao, Yizhe Zhang, Yuyang Wang, Miguel Ángel Bautista, Shuangfei Zhai, Joshua M Susskind, and Jiatao Gu. Starflow2: Bridging language models and normalizing flows for unified multimodal generation. *arXiv preprint arXiv:2605.08029*, 2026.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- DiJia Su, Hanlin Zhu, Yingchen Xu, Jiantao Jiao, Yuandong Tian, and Qingqing Zheng. Token assorted: Mixing latent and text tokens for improved language model reasoning. *arXiv preprint arXiv:2502.03275*, 2025.
- Yao Tang, Li Dong, Yaru Hao, Qingxiu Dong, Furu Wei, and Jiatao Gu. Multiplex thinking: Reasoning via token-wise branch-and-merge. *arXiv preprint arXiv:2601.08808*, 2026.
- Ling Team, Wenting Cai, Yuchen Cao, Chaoyu Chen, Chen Chen, Siba Chen, Qing Cui, Peng Di, Junpeng Fang, Zi Gong, et al. Every sample matters: Leveraging mixture-of-experts and high-quality data for efficient and accurate code llm. *arXiv preprint arXiv:2503.17793*, 2025.
- Qixun Wang, Yang Shi, Yifei Wang, Yuanxing Zhang, Pengfei Wan, Kun Gai, Xianghua Ying, and Yisen Wang. Monet: Reasoning in latent visual space beyond image and language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12030–12040, 2026.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Junhong Wu, Jinliang Lu, Zixuan Ren, Gangqiang Hu, Zhi Wu, Dai Dai, and Hua Wu. Llm are single-threaded reasoners: Demystifying the working mechanism of soft thinking. *arXiv preprint arXiv:2508.03440*, 2025.
- Zhihui Xie, Jiacheng Ye, Lin Zheng, Jiahui Gao, Jingwei Dong, Zirui Wu, Xueliang Zhao, Shansan Gong, Xin Jiang, Zhenguo Li, et al. Dream-coder 7b: An open diffusion language model for code. *arXiv preprint arXiv:2509.01142*, 2025.
- Zhangchen Xu, Yang Liu, Yueqin Yin, Mingyuan Zhou, and Radha Poovendran. Kodcode: A diverse, challenging, and verifiable synthetic dataset for coding. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 6980–7008, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.
- Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhua Chen. Acecoder: Acing coder rl via automated test-case synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12023–12040, 2025.

- Shuangfei Zhai, Ruixiang Zhang, Preetum Nakkiran, David Berthelot, Jiatao Gu, Huangjie Zheng, Tianrong Chen, Miguel Angel Bautista, Navdeep Jaitly, and Josh Susskind. Normalizing flows are capable generative models. *arXiv preprint arXiv:2412.06329*, 2024.
- Ruixiang Zhang, Shuangfei Zhai, Jiatao Gu, Yizhe Zhang, Huangjie Zheng, Tianrong Chen, Miguel Angel Bautista, Joshua Susskind, and Navdeep Jaitly. Flexible language modeling in continuous space with transformer-based autoregressive flows. *Advances in Neural Information Processing Systems*, 38:173273–173323, 2026a.
- Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, and Xin Wang. Soft thinking: Unlocking the reasoning potential of llms in continuous concept space. *Advances in Neural Information Processing Systems*, 38:168990–169012, 2026b.
- Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *Advances in Neural Information Processing Systems*, 38:56729–56762, 2026.
- Yuyan Zhou, Jiarui Yu, Hande Dong, Zhezheng Hao, Hong Wang, Jianqing Zhang, and Qiang Lin. Lepo: Latent reasoning policy optimization for large language models. *arXiv e-prints*, pages arXiv–2604, 2026.
- Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025.
- Zachary Ziegler and Alexander Rush. Latent normalizing flows for discrete sequences. In *International Conference on Machine Learning*, pages 7673–7682. PMLR, 2019.

A Qualitative Analysis of Decoded Latent CoTs

In this appendix, we provide qualitative examples of decoded latent CoTs on HumanEval. Each example contains the raw text decoded from a sampled latent CoT vector and the program generated from the same latent prefix. The decoded latent text is not edited for correctness or fluency. Because this text is reconstructed through the frozen VAE decoder, we do not interpret it as a faithful natural-language explanation of the model’s internal computation. Instead, we use it as a qualitative probe of whether sampled latent trajectories align with coarse algorithmic choices in the final program. These examples show that NF-CoT’s latent variables often correspond to high-level algorithmic choices: different latent samples for the same problem can lead to different implementation strategies while still producing functionally correct programs.

Experimental setup. We use the NF-CoT unified checkpoint with 64 latent tokens. For each task, we sample latent trajectories with guidance scale 1.0, NF temperature 0.9, and answer temperature 0.6. The decoded latent CoT is obtained by converting the unified latent back to the VAE-latent space and greedily decoding it with the VAE decoder. The benchmark subset is from HumanEval and is not part of the training data.

Table 5 Pass counts for the HumanEval tasks considered in the latent-CoT showcase. We provide detailed decoded examples for a representative subset below.

Task	Entry point	Passed / 64
HumanEval/75	<code>is_multiply_prime</code>	55 / 64
HumanEval/124	<code>valid_date</code>	64 / 64
HumanEval/154	<code>cycpattern_check</code>	61 / 64
HumanEval/63	<code>fibfib</code>	64 / 64
HumanEval/44	<code>change_base</code>	64 / 64
HumanEval/38	<code>decode_cyclic</code>	64 / 64
HumanEval/123	<code>get_odd_collatz</code>	58 / 64
HumanEval/89	<code>encrypt</code>	64 / 64

A.1 HumanEval/63: fibfib

Problem

```
def fibfib(n: int):
    """The FibFib number sequence is a sequence similar to the Fibonacci sequence that's defined as follows:
    fibfib(0) == 0
    fibfib(1) == 0
    fibfib(2) == 1
    fibfib(n) == fibfib(n-1) + fibfib(n-2) + fibfib(n-3).
    Please write a function to efficiently compute the n-th element of the fibfib number sequence.
    >>> fibfib(1)
    0
    >>> fibfib(5)
    4
    >>> fibfib(8)
    24
    """
```

This task is a strong showcase because the same problem produces three passing answers with genuinely different implementation strategies: rolling-state dynamic programming, explicit tabulation, and recursive memoization.

Decoded Latent CoT: sample_idx=49, rolling three-state DP

Why useful: The decoded latent CoT discusses compressing the recurrence into fixed state, and the generated answer uses only a, b, and c.

To calculate the Fibonacci number sequence up to the (n) th number, we need to follow these steps:

Plan

1. **Understanding the Fibonacci Sequence**: The Fibonacci sequence is defined as:
 - $F(1) = 0$
 - $F(2) = 1$
 - $F(n) = F(n-1) + F(n-2)$
2. **Compressing the Formula for Efficiency**: We use a fixed amount of space to store the last three Fibonacci numbers and calculate the next. This optimizes, since we don't need to use (n) variables.
3. **Recursive Fibonacci Term**: Calculate the (n) th Fibonacci number:
 - $F(n) = F(n-1) + F(n-2)$
4. **Flow**: This calculation is done in constant $O(1)$ extra space by using three variables.

Generated Answer: sample_idx=49, passed

```
def fibfib(n: int) -> int:
    """Compute the n-th element of the fibfib sequence."""
    # Base cases
    if n == 0:
        return 0
    elif n == 1:
        return 0
    elif n == 2:
        return 1

    # Initialize the first three elements of the sequence
    a, b, c = 0, 0, 1

    # Compute the sequence iteratively up to the n-th element
    for _ in range(3, n + 1):
        # Calculate the next element in the sequence
        next_value = a + b + c
        # Update the previous three elements for the next iteration
        a, b, c = b, c, next_value

    return c
```

Decoded Latent CoT: sample_idx=3, explicit tabulation list

Why useful: The decoded latent CoT discusses tabulation/list storage, and the generated answer appends every FibFib value to fibfib_values.

To solve the Fibonacci number sequence using a tabular approach, we can efficiently compute the n th Fibonacci number and compute the sequence for the first n Fibonacci numbers. The Fibonacci number is defined by the formula:

- $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

Here's a step-by-step breakdown of how we can implement this:

Step-by-Step Explanation

1. **Initialization**:
 - We use a stack (list) to store Fibonacci numbers, starting with the base cases ' $n = 0$ ' and ' $n = 1$ '.
 - This prevents the exponential number of computations and ensures the sequence is computed efficiently.
2. **Base Case**:
 - The first two Fibonacci numbers are known: ' $\text{fib}(0) = 0$ ' and ' $\text{fib}(1) = 1$ '.
3. **Compute the Sequence**:
 - For a given ' n ', compute the value of ' $\text{fib}(n)$ ' by summing the previous two Fibonacci numbers (last and n th).
 - Compute the rest of the sequence numbers: ' $\text{fib}(n-1)$ ', ' $\text{fib}(n-2)$ ', etc., from 3 to ' n '.

4. ****Return Computed Values:****
- Return the array of computed Fibonacci numbers.
5. ****Implementation:**** The function computes the necessary Fibonacci number for 'n', and indeed utilizes efficient tabulation to avoid redundant computations.

Generated Answer: sample_idx=3, passed

```
def fibfib(n: int) -> int:
    """Compute the n-th element of the FibFib sequence."""
    # Base cases
    if n == 0:
        return 0
    if n == 1:
        return 0
    if n == 2:
        return 1

    # Initialize the first three values of the sequence
    fibfib_values = [0, 0, 1]

    # Compute the sequence iteratively up to n
    for i in range(3, n + 1):
        # Calculate the next value as the sum of the last three values
        next_value = fibfib_values[-1] + fibfib_values[-2] + fibfib_values[-3]
        # Append the new value to the list
        fibfib_values.append(next_value)

    # Return the n-th value in the sequence
    return fibfib_values[n]
```

Decoded Latent CoT: sample_idx=35, recursive memoization

Why useful: The decoded latent CoT discusses cache and recursion, and the generated answer uses a nested memoized helper.

To solve this problem, we can use a cache to efficiently compute the nth Fibonacci number. We'll use a recursive function to compute the Fibonacci numbers, but we'll optimize the recursion by storing the results of previous Fibonacci numbers. This method avoids the exponential number of calculations and efficiently fills the cache, leading to a significant efficiency improvement. Here's the detailed version:

Efficient Fibonacci Sequence

Step-by-Step Explanation

1. ****Define the Function:****
- Use a cache (dictionary) to store the value of Fibonacci numbers we have computed.
2. ****Recursive Computation:****
- We compute the Fibonacci number by first calculating the function recursively and store it in the cache.
- Use base cases to reduce complexity.

Generated Answer: sample_idx=35, passed

```
def fibfib(n: int) -> int:
    """Compute the n-th element of the FibFib sequence."""

    # Initialize a dictionary to store the computed values
    memo = {0: 0, 1: 0, 2: 1}

    def helper(x):
        """Recursive helper function with memoization."""
        # Check if the value is already computed
```

```

    if x in memo:
        return memo[x]

    # Compute the value recursively and store it in the memo dictionary
    memo[x] = helper(x - 1) + helper(x - 2) + helper(x - 3)
    return memo[x]

# Call the helper function with the input n
return helper(n)

```

A.2 HumanEval/154: cycpattern_check

Problem

```

def cycpattern_check(a , b):
    """You are given 2 words. You need to return True if the second word or any of its rotations is a
    substring in the first word
    cycpattern_check("abcd","abd") => False
    cycpattern_check("hello","ell") => True
    cycpattern_check("whassup","psus") => False
    cycpattern_check("abab","baa") => True
    cycpattern_check("efef","eeff") => False
    cycpattern_check("himenss","simen") => True

    """

```

This example shows that different latent samples can instantiate the same rotation-substring idea with different surface forms.

Decoded Latent CoT: sample_idx=0, direct rotation loop

Why useful: The decoded latent CoT says to inspect all rotations, and the generated answer loops over rotations with early return.

```

## solve the problem, we need to check for a pattern in a string rotated a certain number of times. The
    first step is to rotate the string and the second step is to inspect all rotations for the given
    pattern.

## Step-by-Step Explanation

1. Understanding Rotations:
    - A rotation of a string is a substring of the original string.
    - For example, if 's = "abcdef"', then the rotations are: 'a., b., c., d., e., f.' and 'b., c., d., e.,
      f., a.'.
2. Checking Rotations:
    - We can generate all possible rotations of a string.
    - For each rotation, we check if it matches the pattern.
3. Rotating and Sorting:
    - This approach can be somewhat complex for all rotations.

```

Generated Answer: sample_idx=0, passed

```

def cycpattern_check(a, b):
    """Check if any rotation of the second word is a substring in the first word."""
    # Get the length of the second word.
    len_b = len(b)

    # Check each rotation of the second word.
    for i in range(len_b):
        # Create a rotation by slicing the string.
        rotation = b[i:] + b[:i]

```

```

    # Check if this rotation is a substring in the first word.
    if rotation in a:
        return True

    # If no rotation is a substring, return False.
    return False

```

Decoded Latent CoT: sample_idx=16, precompute rotations plus any

Why useful: The decoded latent CoT says to generate cyclic permutations, and the generated answer materializes rotations and calls any.

To solve the problem of checking if a word is a substring of any rotation, we need to consider the given function and introduce some checks to check the rotations. Here's how we can implement the function:

Step-by-Step Explanation

1. ****Check the String Rotation:****
 - For each rotation, check if the 'check_string' is a substring. Rotations can be generated by checking if the string is a substring of any of its cyclic permutations.
2. ****Understanding Rotations:****
 - ****Rotations**** are permutations of the string. For any given rotation, if you concatenate the string with itself, the original string becomes a substring.
 - Example: If 's = "abcd"', the rotations are ['"abcd"', '"bcda"', '"cdab"', '"dabc"']'.
3. ****Check Substring Check:****
 - If 'check_string' is a substring of any rotation, it can be checked if it is a substring of 's + s' (the concatenated version of 's').
4. ****Check for No Rotation:****
 - We need to check if the 'check_string' is also a substring of the first rotation. A simple check would be:
 - Check if 'check_string' is a substring of 'a', 'b', 'c', or 'd' for each rotation.
5. ****Implement a Check for All Rotations:****
 - Check all rotations by rotating the string and checking the substring condition.
 - If any rotation contains 'check_string', return 'True'.

Generated Answer: sample_idx=16, passed

```

def cycpattern_check(a, b):
    if not b:
        return True
    if len(b) > len(a):
        return False

    # Generate all rotations of b
    rotations = [b[i:] + b[:i] for i in range(len(b))]

    # Check if any rotation of b is a substring of a
    return any(rot in a for rot in rotations)

```

A.3 HumanEval/89: encrypt

Problem

```

def encrypt(s):
    """Create a function encrypt that takes a string as an argument and
    returns a string encrypted with the alphabet being rotated.
    The alphabet should be rotated in a manner such that the letters
    shift down by two multiplied to two places.
    For example:
    encrypt('hi') returns 'lm'
    encrypt('asdfghjkl') returns 'ewhjklnop'

```

```
encrypt('gf') returns 'kj'
encrypt('et') returns 'ix'
"""
```

This task illustrates another form of latent-controlled implementation diversity: the same Caesar-style cipher is solved either through alphabet-index lookup or ASCII/modulo arithmetic.

Decoded Latent CoT: sample_idx=1, alphabet-index lookup

Why useful: The decoded latent CoT describes alphabet rotation, and the generated answer uses an explicit alphabet string and `.index()`.

To solve the problem of encrypting a string using a Caesar cipher, we'll implement a function that handles the rotation of characters. Here's a step-by-step explanation of how we can achieve this:

Step-by-Step Explanation

1. ****Handle Case**:**
 - Convert the string to handle cases (uppercase and lowercase). This will handle the case where the user strings are partially lowercased.
2. ****Handle Edge Cases**:**
 - Handle edge cases such as empty strings or the handle of digits properly.
3. ****Encoding and Rotation**:**
 - Each character in the string should be shifted by the two-digit number provided.
 - If the shift handles the letters beyond 'z' and 'Z', wrap around to the beginning of the alphabet.
4. ****Handle Edge Handling**:**
 - Repeat the shifted character to handle each character individually.

Generated Answer: sample_idx=1, passed

```
def encrypt(s):
    """Encrypt the string by shifting the alphabet by 4 places."""
    # Define the alphabet
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    # Calculate the shift amount
    shift = 4
    # Initialize an empty result string
    encrypted = ''

    # Iterate over each character in the input string
    for char in s:
        # Check if the character is a lowercase letter
        if char in alphabet:
            # Find the original position of the character in the alphabet
            original_position = alphabet.index(char)
            # Calculate the new position after shifting
            new_position = (original_position + shift) % 26
            # Append the encrypted character to the result
            encrypted += alphabet[new_position]
        else:
            # If it's not a letter, keep the character as is
            encrypted += char

    return encrypted
```

Decoded Latent CoT: sample_idx=23, ASCII modulo arithmetic

Why useful: The decoded latent CoT describes Caesar rotation and wraparound, and the generated answer uses `ord/chr` modulo arithmetic.

To solve the problem of encrypting a string using a Caesar cipher, we need to follow these steps:

1. ****Function Definition**:** Create the function 'rotate_encrypt' that takes two parameters: the 'string'

- and the 'shift'.
2. ****Initialize the Result****: Use a variable to shift each character by the specified number. If the character is shifted by 26, it should wrap around the alphabet.
 3. ****Iterate Through the String****: Iterate through each character in the string and apply the rotation:
 - Shift each letter by the ASCII value of the letter in the opposite alphabet.
 - Use modulo 26 to ensure the shifts are shifted.
 4. ****Encrypt the Character****:
 - Initialize the function to add a shifted character to the result.
 5. ****Return the Encrypted String****: Return the encrypted string.

Generated Answer: sample_idx=23, passed

```
def encrypt(s):
    """Encrypts the input string by rotating the alphabet by 4 positions."""
    encrypted_string = []

    for char in s:
        if 'a' <= char <= 'z': # Check if the character is a lowercase letter
            # Rotate the character by 4 positions
            new_char = chr((ord(char) - ord('a') + 4) % 26 + ord('a'))
            encrypted_string.append(new_char)
        elif 'A' <= char <= 'Z': # Check if the character is an uppercase letter
            # Rotate the character by 4 positions
            new_char = chr((ord(char) - ord('A') + 4) % 26 + ord('A'))
            encrypted_string.append(new_char)
        else:
            # Non-alphabet characters are not changed
            encrypted_string.append(char)

    return ''.join(encrypted_string)
```

Takeaway. Across these examples, decoded latent CoTs are imperfect at the token level but consistently reflect the coarse algorithmic route followed by the final program. The `fibfib` examples are especially illustrative: the same problem yields rolling-state dynamic programming, explicit table-based dynamic programming, and recursive memoization. This supports the view that NF-CoT does not merely store a single hidden solution trace. Instead, its latent space captures a distribution over plausible reasoning trajectories, allowing different samples to steer generation toward distinct but correct implementations.

B Experimental Setup

We evaluate two variants of our latent-reasoning normalizing flow (NF): the *dual-path* model, in which the backbone LLM is invoked twice per training step (once for flow-density estimation and once for cross-entropy answer prediction), and the *unified-path* model, in which a single causal forward computes both losses. Both variants share the same backbone, latent geometry, dataset, two-stage training schedule, and decoding pipeline, and differ only in the conditioning sequence and which components are trainable in Stage 1.

B.1 Dual-Path Model

The dual-path model forwards the backbone twice per gradient step. The two paths use *distinct* latent-to-hidden projectors (`flow_projector` for the flow path, `vae.latent_to_decoder` for the CE path) and *distinct* boundary tokens (BOT and AE). Optional diffusion-forcing on the CE path is supported but disabled in our reported runs.

B.2 Unified-Path Model

The unified model fuses the two paths into a single causal sequence

$$[\text{prompt}; \text{BOT}; \text{flow_proj}(\tilde{x}); \text{answer}],$$

where $\tilde{x} = \pi_{\text{deep}}(\text{shallow}(x + \epsilon))$ is the latent in post-shallow, post-permutation space and $\epsilon \sim \mathcal{N}(0, \sigma_{\text{dq}}^2 I)$ is dequantization noise (the *same* noise drives NF and CE). A single backbone forward yields the hidden states used to compute the NF NLL on positions $[T_p, T_p + N)$ via `output_head` and the CE loss on positions $[T_p + N, T_p + N + T_a)$ via the LM head. Because the shallow MetaBlocks are invertible, conditioning the CE path on $\tilde{x}$ is representationally equivalent to conditioning on raw latents while eliminating the second backbone forward; the unified model therefore needs *only one* latent-to-hidden projector and *one* prefix marker.

B.3 Training Hyperparameters

Backbone and latent geometry. Both variants use `Qwen3-8B-Base` as the backbone. The latent sequence has $N = 64$ slots of dimension $D = 2560$, identical to the backbone hidden size. Latents are produced by a frozen transformer-encoder VAE with encoder `Qwen3-4B-Base`; the VAE encoder is never updated. Dequantization noise has $\sigma_{\text{dq}} = 0.3$. The flow-density head consists of an MLP projector (LayerNorm $\rightarrow$ Linear $\rightarrow$ GELU $\rightarrow$ Linear), five MetaBlocks [Gu et al. \(2026a\)](#) with channel width 2048, two layers per block, head dim 64, alternating FLIP/IDENTITY permutations (so the top, deep, block is IDENTITY and matches the LLM’s L2R causal mask), and a zero-init `NFOutputHead` that emits affine parameters (σ, μ) with softplus-positivized scale and tanh soft-clip at ± 4.0 .

Loss. The training objective is $\mathcal{L} = \lambda_{\text{NF}} \mathcal{L}_{\text{NF}} + \lambda_{\text{CE}} \mathcal{L}_{\text{CE}}$ with $\lambda_{\text{NF}} = \lambda_{\text{CE}} = 1$. The flow-NLL has the standard form $\mathcal{L}_{\text{NF}} = \frac{1}{2} \mathbb{E} \|z\|_2^2 - \mathbb{E} \sum_k \log \det J_k$, and the CE term is teacher-forcing cross-entropy on answer tokens with PAD positions masked out. Classifier-free guidance is supported via a learned `null_embed`; the prompt is dropped to `null_embed` with probability $p_{\text{drop}} = 0.1$ (batch-level for the dual-path model, per-sample for the unified-path model).

Two-stage curriculum. We follow a frozen-backbone $\rightarrow$ joint-finetune curriculum:

- **Stage 1 (latent-projector warm-up).** The backbone is frozen. The shallow blocks *are* trained in Stage 1. Stage 1 uses 100 000 samples, 1 epoch, per-device batch size 2, gradient accumulation 4 (global batch 32 on 4 GPUs), peak learning rate 1×10^{-4} with 100 linear-warmup steps.
- **Stage 2 (joint finetuning).** The backbone and shallow blocks are unfrozen and the full Python subset of the dataset is used for 2 epochs. We use per-device batch size 1, gradient accumulation 8 (global batch 64 on 8 GPUs), peak learning rate 5×10^{-5} , with 200 linear-warmup steps.

For both stages we use AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.01, gradient clip 1.0, cosine learning-rate decay, `bfloat16` mixed precision, and `FSDP hybrid_shard` with auto-wrap on `Qwen3DecoderLayer`, backward/forward prefetching, CPU-RAM-efficient loading, and activation checkpointing. Training is run on NVIDIA DGX B200 nodes (4 GPUs in Stage 1, 8 GPUs in Stage 2). The backbone weights for Stage 2 come directly from the pre-trained `Qwen3-8B-Base` checkpoint. Training uses the HuggingFace Trainer default seed when no explicit seed is specified in the configuration. The total number of optimizer steps is determined by the epoch budgets, global batch sizes, and filtered training split rather than by a fixed `max_steps` override.

Hyperparameter table. Table 6 summarizes the hyperparameters of both variants.

B.4 Inference Configuration

For the main benchmark evaluations, inference is split into two phases per evaluation example. Phase 1 generates latent thoughts via the NF reverse pass; Phase 2 produces an answer by feeding the latent prefix to the backbone as soft-token `input_embeds`. The speed ablation in Table 2 uses vLLM answer decoding as stated there; this subsection describes the main benchmark decoding pipeline.

Phase 1: latent generation (PyTorch). We sample $z \sim \mathcal{N}(0, T_z^2 I) \in \mathbb{R}^{N \times D}$, prefix the prompt embeddings with the model’s boundary token (BOT for both variants; the dual-path model adds AE only on the CE path), and run an autoregressive NF reverse pass through the LLM with KV-cache reuse: at each step t , the LLM hidden at the latest position is mapped through `output_head` to obtain (σ_t, μ_t) , the next latent is computed

Table 6 Hyperparameters for the dual-path and unified-path NF models. Shared settings are shown once across both model columns.

Setting	Dual-path	Unified-path
Backbone LLM	Qwen3-8B-Base	
Frozen latent encoder	TransEncoderVAE with Qwen3-4B-Base encoder	
Latent geometry	$N = 64$ slots, latent dimension $D = 2560$	
Shallow MetaBlocks	5 blocks, trained in Stage 1	5 blocks, trained in Stage 1
Shallow block width	2048 channels, 2 layers per block, head dimension 64	
Deep block	Causal left-to-right LLM block	
Soft-clip / dequantization	Soft-clip 4.0; dequantization noise $\sigma_{dq} = 0.3$	
Loss weights	$\lambda_{NF} = 1.0$, $\lambda_{CE} = 1.0$	
Prompt dropout	$p_{drop} = 0.1$ at batch level	$p_{drop} = 0.1$ per sample
Latent-to-hidden projector	<code>flow_projector</code> plus W_{dec}	Shared <code>flow_projector</code>
Prefix marker(s)	BOT, AE	BOT
Stage-1 frozen modules	Backbone and shallow blocks	Backbone and shallow blocks
Stage-2 trainable modules	All model parameters except the VAE encoder	
Optimizer	AdamW, $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.01, gradient clip 1.0	
Learning rate / warmup	Stage 1: 1×10^{-4} with 100 warmup steps; Stage 2: 5×10^{-5} with 200 warmup steps	
Schedule	Cosine decay	
Batch configuration	Per-device batch 2 / 1 and gradient accumulation 4 / 8 for Stage 1 / Stage 2	
Hardware	4 / 8 NVIDIA B200 GPUs for Stage 1 / Stage 2	
Epochs / Stage-1 budget	Stage 1: 1 epoch over 100,000 samples; Stage 2: 2 epochs	
Precision / sharding	bfloat16; FSDP <code>hybrid_shard</code> with activation checkpointing	
Random seed	HuggingFace Trainer default when unset in the configuration	

as $x_t = \sigma_t z_t + \mu_t$, the projected latent `flow_proj( $x_t$ )` is appended, and the LLM is advanced by one position over the cached keys/values. After $N = 64$ steps, the deep permutation is inverted; for the dual-path model we additionally invert the five shallow MetaBlocks to recover raw latents in encoder space, whereas the unified model keeps the latent in $\tilde{x}$ space (post-shallow, post-permute) since the LLM was trained to read it there. Classifier-free guidance is applied in (σ, μ) space following STARFlow Gu et al. (2026a).

Phase 2: answer decoding (vLLM). The full backbone weights are extracted from the trained checkpoint and loaded into an offline `vllm.LLM` engine with `enable_prompt_embeds=True` (required for embedding inputs). For each problem we feed the soft prefix [`prompt_embeds`; `marker`; `proj(latents)`] through the engine’s `prompt_embeds` interface and sample answer tokens. The marker and projector are `AE/ $W_{dec}$` for the dual-path model and `BOT/flow_projector` for the unified model.

Decoding hyperparameters. For each benchmark problem, we draw 16 independent samples and report `pass@1` as the average single-sample correctness over the 16 candidates. Each sample uses an independent NF noise draw (no fixed seed at inference). For Table 1, we use this standard $n = 16$ sampling configuration unless otherwise specified. For Figure 3, we run a separate `pass@k` scaling evaluation with up to 128 sampled candidates per problem. The `pass@1` endpoints in that scaling run are reported from the same candidate pool used for the `pass@k` curves and are not used to compute Table 1. Decoding uses temperature $T_a = 0.6$, top- $p = 1.0$, and a maximum of 1024 new tokens; NF sampling temperature and CFG strength are *variant-specific* and were tuned on a held-out validation slice:

- **Dual-path:** $T_z = 1.0$, $w_{CFG} = 3.5$.
- **Unified-path:** $T_z = 0.9$, $w_{CFG} = 1.0$; a moderately under-temperated NF sampler is preferred for this model.

B.5 Execution-Guided RL Details

Overview. We further apply execution-guided RL to the supervised NF-CoT (Unified) checkpoint. During RL, the VAE encoder/decoder shell, shallow flow blocks, flow projector, affine output head, and auxiliary latent-control embeddings are kept frozen. We update only the shared backbone LLM, which is used both as the autoregressive block of the normalizing flow and as the answer decoder. Thus, the same trainable parameters receive reward-driven updates from both the latent-trajectory policy and the answer-token policy.

Policy factorization. Each rollout consists of a latent trajectory $\tilde{x} = (\tilde{x}_1, \dots, \tilde{x}_N)$ with $N = 64$, followed by an answer token sequence $y = (y_1, \dots, y_L)$. We view both the continuous latent trajectory and the answer tokens as actions. The latent density is computed by inverting the affine transformation predicted by the flow block at each latent position. For the realized latent $\tilde{x}_t$, the model predicts affine parameters (x_a^t, x_b^t) and recovers the corresponding Gaussian noise as

$$z_t = \frac{\tilde{x}_t - x_b^t}{x_a^t}. \quad (\text{B.1})$$

The latent log-probability is then

$$\log p_\theta(\tilde{x}_t \mid p, \tilde{x}_{<t}) = -\frac{1}{2}\|z_t\|_2^2 - \sum_{d=1}^D \log |x_{a,d}^t|, \quad (\text{B.2})$$

up to the constant Gaussian normalization term, which cancels in the PPO ratio. Since the shallow flow blocks are frozen and the action is defined in post-shallow latent space, their log-determinants are constant with respect to the trainable parameters and are omitted from the RL objective.

RL objective. For each prompt, we sample a group of $n = 8$ rollouts and compute GRPO advantages from their execution rewards:

$$\hat{A}_i = \frac{R_i - \mu}{\sigma + \varepsilon_\sigma}, \quad (\text{B.3})$$

where μ and σ are the group mean and standard deviation. The final RL loss combines a standard token-level PPO objective with a sequence-level latent PPO objective:

$$\mathcal{L}_\theta = \mathcal{L}_\theta^{\text{tok}} + \mathcal{L}_\theta^{\text{lat}}. \quad (\text{B.4})$$

The token term is computed on answer tokens only. The latent term applies the same clipped PPO surrogate to the likelihood ratio of the complete latent trajectory. Specifically, we define the per-step latent log-ratio as

$$\delta_{i,t}^{\text{lat}} = \log p_\theta(\tilde{x}_{i,t} \mid p_i, \tilde{x}_{i,<t}) - \log p_{\theta_{\text{old}}}(\tilde{x}_{i,t} \mid p_i, \tilde{x}_{i,<t}), \quad (\text{B.5})$$

$$\Delta_i^{\text{lat}} = \sum_{t=1}^N \delta_{i,t}^{\text{lat}}, \quad (\text{B.6})$$

$$r_i^{\text{lat}} = \exp(\text{clip}(\Delta_i^{\text{lat}}, -20, 20)). \quad (\text{B.7})$$

Then the latent PPO term is

$$\begin{aligned} \mathcal{L}_\theta^{\text{lat}} = & -\mathbb{E}_i \left[\min \left(r_i^{\text{lat}} \hat{A}_i, \bar{r}_i^{\text{lat}} \hat{A}_i \right) \right] \\ & + \beta_{\text{KL}} \mathcal{K}_i^{\text{lat}}, \end{aligned} \quad (\text{B.8})$$

where

$$\bar{r}_i^{\text{lat}} = \text{clip}(r_i^{\text{lat}}, 1 - \varepsilon, 1 + \varepsilon). \quad (\text{B.9})$$

Here $\varepsilon = 0.2$, and $\mathcal{K}_i^{\text{lat}}$ is a sampled log-ratio penalty between the actor and the frozen reference policy on the same realized latent trajectory. We use the same KL coefficient for the token and latent terms.

Optimization and evaluation. We train for 150 RL steps with AdamW using learning rate 3×10^{-6} , batch size 64 prompts, and 8 rollouts per prompt. The KL coefficient is $\beta_{\text{KL}} = 10^{-3}$, and the PPO clip range is 0.2. During RL rollout, we use answer temperature 0.9 and latent temperature 1.0.

After RL, we evaluate using the same decoding protocol as the supervised NF-CoT models: answer temperature 0.6, NF temperature 0.9, classifier-free guidance scale 1.0, and 16 samples per problem for HumanEval, HumanEval+, MBPP, and MBPP+, LiveCodeBench v6.

Table 7 Backbone drift relative to the original Qwen3-8B-Base checkpoint. The overall drift difference is small, but the stage-2-only run shows substantially larger drift in the final transformer layer.

Metric	Stage 1+2	Stage-2-only	Diff.
Overall rel-L2	0.1574	0.1599	1.016×
Overall 1 - cos	0.01237	0.01278	1.033×
Layer 0 rel-L2	0.142	0.144	+1%
Layer 18 rel-L2	0.144	0.144	0%
Layer 35 rel-L2	0.121	0.143	+18%
Embed/head/norm rel-L2	0.261	0.261	0%

C Additional Analysis

C.1 Backbone Drift Diagnostic

To further diagnose the effect of the frozen-backbone warm-up stage, we compare the default two-stage curriculum against a stage-2-only variant that directly starts full-parameter joint training from randomly initialized flow components. We analyze both early training dynamics and the final backbone drift relative to the original Qwen3-8B-Base checkpoint.

Early training dynamics. The two runs reach similar final optimization losses, but they differ substantially at the beginning of joint training. In the default curriculum, the flow-side components have already been trained while the backbone is frozen. At the start of stage 2, the model has $L_{\text{NF}} \approx -0.42$ and log-determinant approximately -0.92 , indicating that the shallow flow blocks have moved away from an identity map and already provide a nontrivial density model over the latent targets.

By contrast, the stage-2-only run starts joint training with $L_{\text{NF}} \approx 0.47$ and log-determinant close to zero. This is close to a random Gaussian-NLL baseline and suggests that the shallow blocks initially behave almost like an identity map. Its initial gradient norm is also substantially larger than that of the warm-started run, 1.96 versus 0.96. These early dynamics suggest that, without the warm-up stage, the pretrained backbone is immediately exposed to large gradients from a poorly calibrated NF branch.

Final backbone drift. Table 7 shows that the stage-2-only model does not simply drift uniformly farther from the pretrained model. Its global relative L2 distance is only 1.6% larger than the default curriculum, and the global cosine drift increases by only 3.3%. However, the layerwise pattern is much more concentrated: the final transformer layer has 18% larger relative L2 drift, while the embedding layer, the middle layer, and the embedding/head/normalization parameters are nearly unchanged.

The final transformer layer is the shared representation that directly feeds both the LM head and the NF output head. In the stage-2-only run, this layer is exposed to early NF gradients before the flow projector, shallow blocks, and output head have learned a meaningful latent-density interface. As a result, the largest additional movement appears exactly at the layer most directly coupled to the randomly initialized NF branch. The warm-up stage mitigates this effect by first stabilizing the flow-side components while the backbone is frozen, so that the subsequent joint-finetuning stage sends a more calibrated NF learning signal into the pretrained backbone.

C.2 Pairwise Output Diversity

A key goal of latent-CoT sampling is not only to generate multiple lexically different completions, but to explore genuinely different algorithmic solutions. We therefore measure within-prompt output diversity among passing programs under matched sampling budgets. For four randomly selected HumanEval problems (HumanEval/154, HumanEval/124, HumanEval/38, and HumanEval/123), we draw 64 samples per problem from NF-CoT and from the Qwen3-8B-Base teacher. For NF-CoT, we use the same decoding configuration as the main evaluation, with NF temperature $\tau_z = 1.0$ and answer temperature $T = 0.6$; for the base model, we use the same answer temperature $T = 0.6$ and the same wrapped prompt. We keep only samples that pass the HumanEval unit tests, and then uniformly sample $k = 8$ passing programs per problem without

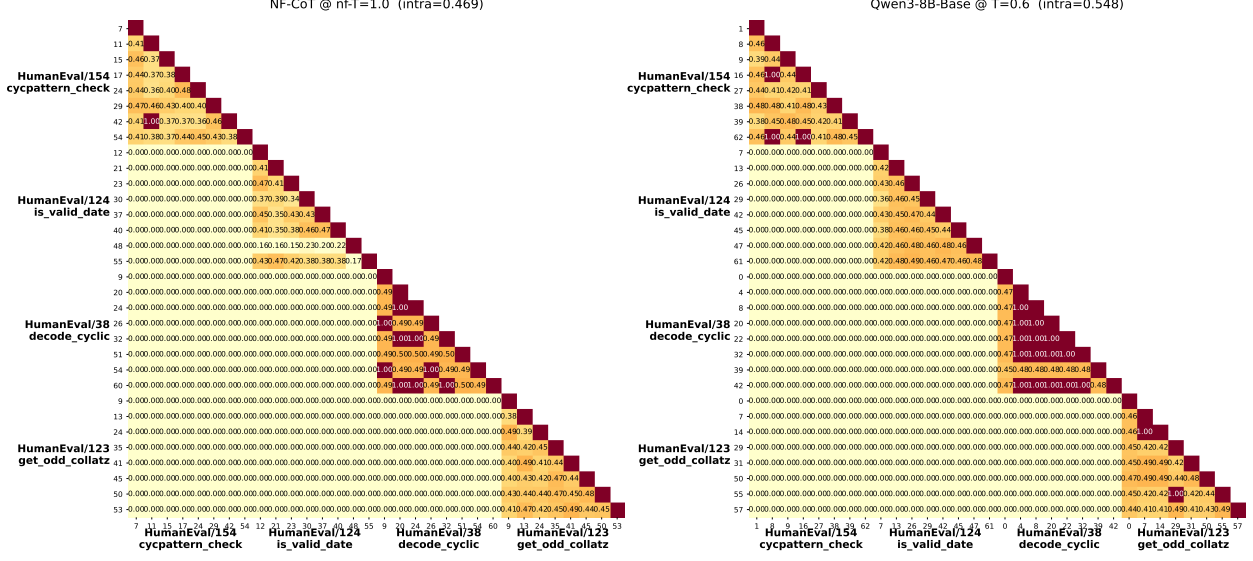


Figure 6 Pairwise output similarity among passing HumanEval programs. Each panel contains $32 = 4 \times 8$ passing samples from four HumanEval problems, with intra-task blocks on the diagonal and cross-task entries set to zero to isolate within-prompt structure. Similarity is computed using a structure-aware AST metric; lower values indicate higher diversity. NF-CoT achieves lower mean intra-prompt similarity than the Qwen3-8B-Base teacher, indicating more structurally diverse passing solutions under the same answer temperature.

replacement. To avoid reporting a fortuitous subset, we repeat this sub-sampling over 64 random seeds and visualize the seed whose mean intra-prompt similarity is closest to the across-seed median.

Surface-level similarity metrics can be misleading in this setting. Embedding cosine on raw generations, character n -gram similarity, and edit distance are heavily affected by natural-language reasoning text, variable names, formatting, and whitespace. At temperature $T = 0.6$, the base model often produces many lexically different paraphrases of the same underlying algorithm, which can spuriously inflate apparent diversity under token-level metrics. We therefore use a structure-aware metric defined on the entry-point function body:

$$s(a, b) = \frac{1}{2} \cos(\phi_{2g}(a), \phi_{2g}(b)) + \frac{1}{2} \mathbb{I}[h(a) = h(b)].$$

Here, $\phi_{2g}(\cdot)$ is the count vector of parent-child AST-node-type 2-grams, which captures control-flow and syntactic patterns such as loop updates, list construction, calls, and conditional branches. The function $h(\cdot)$ is the hash of a canonicalized AST after local variables are α -renamed to canonical identifiers and docstrings/comments are stripped. The cosine term gives a smooth measure of structural similarity between non-identical programs, while the hash term pins truly identical canonical programs to similarity 1. Both terms are invariant to variable renaming, formatting, and comments.

Figure 6 shows that NF-CoT produces more structurally diverse passing solutions than the base model. The mean intra-prompt similarity drops from 0.548 for Qwen3-8B-Base to 0.469 for NF-CoT, a relative reduction of approximately 14%. The improvement is consistent across all four tasks: the intra-prompt similarity decreases by 0.017 on HumanEval/154, 0.156 on HumanEval/124, 0.007 on HumanEval/38, and 0.002 on HumanEval/123. The largest difference appears on HumanEval/124 (`is_valid_date`), where the base model largely collapses onto a canonical split-and-branch implementation, whereas NF-CoT samples a wider mixture of structurally different solutions, including regular-expression-based, exception-handling-based, and date-library-based implementations.

These results suggest that the latent flow does more than induce surface-level variation. The base model is already lexically diverse under temperature sampling, but its structural distribution remains peaked around a small number of common implementation templates. NF-CoT injects stochasticity upstream of token generation by sampling from the latent prior $z \sim \mathcal{N}(0, \tau_z^2 I)$, so different latent trajectories can steer decoding

toward different algorithmic regions before answer tokens are generated. As a result, latent sampling changes which solution strategy the model follows, rather than merely changing how the same strategy is verbalized or formatted. We use the same decoding configuration as our main evaluation, with $\tau_z = 1.0$ and answer temperature $T = 0.6$, so the diversity comparison is aligned with the reported evaluation setting rather than tuned specifically for this analysis.